Aprendizaje por Refuerzo

Programación Dinámica

Luis Daniel Hernández Molinero
Ciencia de la Computación e Inteligencia Artificial
Dept. Ingeniería de la Información y las Comunicaciones

Universidad de Murcia

Máster en Inteligencia Artificial

2025 / 2026

Índice de Contenidos

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Desarrollo

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

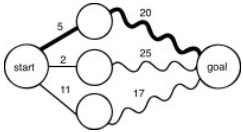
6. Referencias

Introducción a la programación dinámica

- **¿Qué es la programación dinámica?** (Bellman et al., 1957)
  - **Dinámica:** componente secuencial o temporal del problema.
  - **Programación:** optimización de un "programa", como programación matemática.
- **Un método** para resolver problemas complejos:
  - **Descomponerlo** en subproblemas.
  - **Resolver** los subproblemas.
  - **Combinar** las soluciones de los subproblemas.
- **Ventajas** de la programación dinámica
  - **Reutilización de subproblemas resueltos:** Evita resolver el mismo subproblema.
  - **Optimalidad garantizada:** Si el problema cumple ciertas propiedades.
  - **Aplicabilidad:** Se aplica a una amplia gama de problemas, como secuencias, optimización o grafos.

Requisitos para la Programación Dinámica

- **La programación dinámica es** es un método general para problemas con **dos propiedades**:
  - **Estructura óptima de subproblemas:**
    - Aplica el **principio de optimalidad**.
    - La solución óptima se obtiene de los óptimos de los subproblemas.
  - **Subproblemas superpuestos:**
    - Los subproblemas se repiten muchas veces.
    - Memoización y Tabulación: Las soluciones pueden almacenarse y reutilizarse en un orden *ascendente*.



- **Los procesos de decisión de Markov (MDP) satisfacen ambas propiedades:**
  - La ecuación de Bellman presenta una estructura óptima de subproblemas.
  - La función de valor almacena y reutiliza soluciones.

Planificación mediante programación dinámica

- Diferencias clave entre **planificación** y **aprendizaje por refuerzo** (completo).

Aspecto	Planificación	Aprendizaje por refuerzo completo
Conocimiento del MDP	Se conoce el MDP completo (P, R).	El MDP es desconocido; el agente lo aprende al interactuar.
Exploración	No es necesaria: el modelo ya es conocido.	Es necesaria: el agente debe explorar para aprender el entorno.
Técnicas principales	Programación dinámica, iteración de valor, iteración de política.	Q-learning, SARSA, aprendizaje profundo.
Objetivo	Resolver el MDP conocido (evaluar o controlar).	Aprender la política óptima y las dinámicas del entorno.

- La Programación Dinámica se utiliza para la **planificación** en un MDP:
  - Para **predicción**:
    - Entrada: MDP $\langle S, A, P, R, \gamma \rangle$ y política π.
    - Salida: función de valor v_π: $v_\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$.
  - Para **control**:
    - Entrada: MDP $\langle S, A, P, R, \gamma \rangle$.
    - Salida: función de valor óptima v_* y política óptima π^*: $v_*(s) = \max_\pi \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$.
  - La programación dinámica asume **conocimiento completo** del MDP. Ej: Cubo Rubik vs Conducir

Desarrollo

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Evaluación Iterativa de Políticas

- **Problema (Predicción):** evaluar una política dada π. $v_\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s\right]$.
- Podemos usar la expresión de las **ecuaciones de Bellman**:

$$v_\pi(s) = \sum_{a\in A} \pi(a|s)\left(R_s^a + \gamma \sum_{s'\in S} P_{ss'}^a v_\pi(s')\right) \qquad v_\pi(s) = \sum_{a\in \mathcal{A}} \pi(a|s) \sum_{s',r} P_{ss'}^{ar}\left(R_s^a + \gamma v_\pi(s')\right)$$

- Tiene una expresión **matricial**: $\mathbf{v}_\pi = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \cdot \mathbf{v}_\pi$
  - $\mathbf{v}_\pi = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1}\mathbf{R}^\pi$ (si invertible)
  - Existencia y unicidad de v_π si $\gamma < 1$ o temine
- **Otra solución:** $\mathbf{v_1} \to \mathbf{v_2} \to \ldots \to \mathbf{v_k} \to \ldots \xrightarrow{?} \mathbf{v}_\pi$
- **Actualizaciones basadas en expectativas** (o actualizaciones esperadas): $\mathbf{v}_{k+1}^\pi = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{v}_k$

$$v_{k+1}(s) = \sum_{a\in A} \pi(a|s)\left(R_s^a + \gamma \sum_{s'\in S} P_{ss'}^a v_k(s')\right) \qquad v_{k+1}(s) = \sum_{a\in \mathcal{A}} \pi(a|s) \sum_{s',r} P_{ss'}^{ar}\left(R_s^a + \gamma v_k(s')\right)$$

- v_π es un punto fijo → Para v_π la actualización esperada es la ecuación de Bellman
  - Teorema: $\lim_{k\to\infty} v_k = v_\pi$
- ¡Tenemos algoritmo! Se llamará **evaluación iterativa de políticas**.

Algoritmo: Evaluación iterativa de políticas

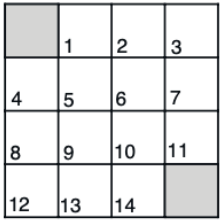
Algorithm 1 Evaluación Iterativa de Políticas para Estimar $V \to v_\pi$

Entrada: π, la política a evaluar. $\epsilon > 0$, un pequeño umbral, que determina la precisión de la estimación.

INICIALIZAR $V(s)$ arbitrariamente para $s \in S$, y $V(\text{terminal}) = 0$.
repeat
  $\Delta \leftarrow 0$
  for $s \in S$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
    $\Delta \leftarrow \text{máx}(\Delta, |v - V(s)|)$
  end for
until $\Delta < \epsilon$ # $\Delta = \text{máx}_{s \in S} |v_{k+1}(s) - v_k(s)|$

Ejemplo: Pequeño mundo en cuadrícula
Planteamiento del Problema

actions



$R_t = -1$
on all transitions

- Estados no terminales: 1, ..., 14.
- Un estado terminal (mostrado dos veces como cuadrados sombreados).
- Función de recompensa esperada: $r(s, a, s') = -1$ hasta que se alcance el estado terminal.
- MDP episódico no descontado ($\gamma = 1$).
- Las acciones que llevan fuera de la cuadrícula dejan el estado sin cambios.
  P.e, $p(6, -1|5, \text{derecha}) = 1$, $p(7, -1|7, \text{derecha}) = 1$, y $p(10, r|5, \text{a}) = 0$ para todo $r \in R$ y $a \in A$.
- El agente sigue una política aleatoria equiprobable:

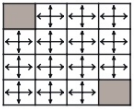
$$\pi(norte|\cdot) = \pi(este|\cdot) = \pi(sur|\cdot) = \pi(oeste|\cdot) = 0{,}25$$

Ejemplo: Pequeño mundo en cuadrícula
Evaluación Iterativa de Políticas

v_k for the random policy | greedy policy w.r.t. v_k

$k = 0$

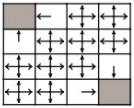
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0



random policy

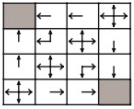
$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0



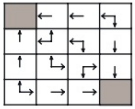
$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



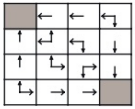
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



$k = 10$

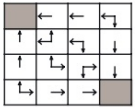
0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



optimal policy

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



$$v_1(s) = \sum_{a \in A} \pi(a|s) \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_0(s') \right]$$

$$= 4 \cdot \frac{1}{4} \cdot [-1 + 0] = -1$$

$$v_2(s) = 0{,}25\,[-1 + v_1(s_{\text{Norte}})] + 0{,}25\,[-1 + v_1(s_{\text{Sur}})] + 0{,}25\,[-1 + v_1(s_{\text{Este}})] + 0{,}25\,[-1 + v_1(s_{\text{Oeste}})] = -2$$

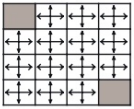
$$v_2(s_T) = 0{,}25\,[-1 + 0] + 3 \times 0{,}25\,[-1 + (-1)] = -1{,}75$$

Ejemplo: Pequeño mundo en cuadrícula
¿Por qué mejora la política?

v_k for the random policy | greedy policy w.r.t. v_k

$k = 0$

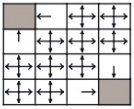
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0



random policy

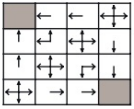
$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0



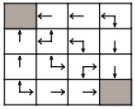
$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



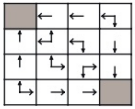
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



$k = 10$

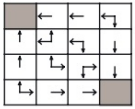
0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



optimal policy

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



$$Q_k(s,a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_k(s')$$
$$= R_s^a + v_k(s')$$
$$(\gamma = 1 \text{ y } P_{ss'}^a = 1)$$

s adyacente a $s_T \Rightarrow Q_2(s,a) = R_s^a + v_2(s_T) = -1 + 0 = -1$ (opt.)

$\Rightarrow Q_2(s,a) = R_s^a + v_2(s') = -1 - 2 = -3$

s esquina a $s_T \Rightarrow Q_2(s,a) = R_s^a + v_2(s_T) = -1 - 1{,}7 = -2{,}7$ (opt.)

$\Rightarrow Q_2(s,a) = R_s^a + v_2(s') = -1 - 2{,}0 = -3$

Desarrollo

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Mejora de Política

Teorema (Teorema de Mejora de Políticas Determinísiticas)

Sean π y π' dos política que verifican $q_\pi(s, \pi'(s)) \geq v_\pi(s)\ \forall s \in S$, entonces:
la política π' debe ser tan buena, o mejor que, π: $v_{\pi'}(s) \geq v_\pi(s)$

- Definimos una nueva **política determinística** o greedy: $\pi'(s) = \arg\max_a q_\pi(s, a)$, por tanto verifica:
$$\pi'(s) = \arg\max_a q_\pi(s, a) = \arg\max_a \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_\pi(s')\right]$$
- ¿**Será mejor** esta política determinística? **Sí**, porque:
  - $q_\pi(s, \pi'(s)) = \max_{a \in A} q_\pi(s, a) \Longrightarrow q_\pi(s, \pi'(s)) \geq q_\pi(s, a), \quad \forall a \in A$
  - $v_\pi(s) = \sum_{a \in A} \pi(a|s) q_\pi(s, a) \leq \max_{a \in A} q_\pi(s, a) = q_\pi(s, \pi'(s))$
  - Por el teorema de mejora de políticas: $v_{\pi'}(s) \geq v_\pi(s)$ (es mejor)
- Además: $v_{\pi'}(s) = \max_{a \in \mathcal{A}} q_\pi(s, a) = \max_{a \in \mathcal{A}} \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_\pi(s')\right]$, por ser codiciosa sobre $q_\pi()$
- **Durante el proceso:** llegamos a π, calculamos la nueva política codiciosa π' y **ocurre que** $v_\pi = v_{\pi'}$, *pues*:
  - Se cumple la igualdad: $v_{\pi'}(s) = \max_{a \in \mathcal{A}} q_\pi(s, a) = \max_{a \in \mathcal{A}} \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_{\pi'}(s')\right]$
  - Que se corresoponderá con **la ecuación de optimalidad de Bellman**: π y π' son π_*.
- **Generalizando** a $\pi(a|s)$ el T. de Mejora de Políticas se aplica tal cual. Si hay empates, distribuir π'.

Desarrollo

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Profundizamos en ...

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Iteración de políticas

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_*,$$

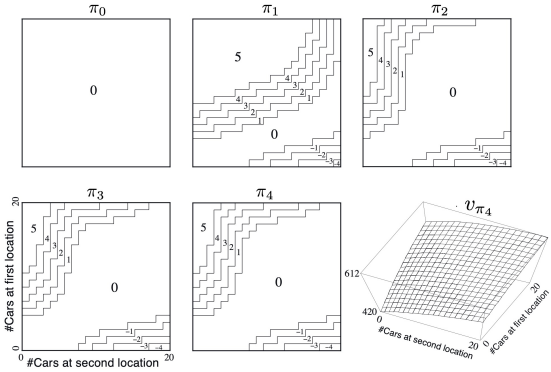
Algorithm 2 Iteración de políticas (usando evaluación iterativa de políticas) para estimar $\pi \to \pi^*$

1. **Inicialización.** $V(s) \in \mathbb{R}$ y $\pi(s) \in \mathcal{A}(s)$ arbitrariamente para todo $s \in \mathcal{S}$; $V(\text{terminal}) = 0$
2. **Evaluación de la política.** Estimar v_π mediante evaluación iterativa de políticas. **Algoritmo 1**
3. **Mejora de la política.** Generar $\pi' \geq \pi$ mediante una mejora codiciosa de la política.

```
policy-stable ← True
for s ∈ S do
    old-action ← π(s)
    π(s) ← arg máx_a Σ_{s',r} p(s', r|s, a)[r + γV(s')]
    if old-action ≠ π(s) then
        policy-stable ← False
    end if
end for
if policy-stable then
    detener y devolver V → v* y π → π*; de lo contrario, volver a 2.
end if
```

Ejemplo: Alquileres de coches

- **Estados:** Dos ubicaciones, con un máximo de 20 autos en cada una.
- **Acciones:** Mover hasta 5 autos entre las ubicaciones durante la noche.
- **Recompensa:** \$10 por cada auto alquilado (debe estar disponible). -\$2 por coche desplazado.
- **Transiciones:** Los autos son devueltos y solicitados de forma aleatoria.
  - Distribución de Poisson: n devoluciones/solicitudes con probabilidad $P(\lambda) = \frac{\lambda^n}{n!} e^{-\lambda}$.
  - Primera ubicación: solicitudes promedio $= 3$, devoluciones promedio $= 3$.
  - Segunda ubicación: solicitudes promedio $= 4$, devoluciones promedio $= 2$.



Iteración de Políticas Modificada

- **Iteración de Políticas Clásica**: Repite los siguientes pasos 1 y 2 hasta que no cambia π
  1. **Evaluación de políticas:** $v_\pi(s) = \sum_{a \in A} \pi(a \mid s) \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_\pi(s') \right]$.
     - $v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right)$
     - Teorema: $\lim_{k \to \infty} v_k = v_\pi$
     - En la práctica $|v_{k+1} - v_k| < \epsilon$
  2. **Mejora de políticas (greedy):** $\pi'(s) = \arg\max_a \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_\pi(s') \right]$.
- **Algunas preguntas**. En algunos problemas llagamos antes a la política óptima, entonces ...
  - ¿Es necesario que la evaluación de políticas converja completamente a v_π?
  - ¿O debería introducirse una condición de parada (e.g., convergencia ϵ de la función de valor)?
  - ¿O simplemente detenerse después de k iteraciones de evaluación iterativa de políticas?
- Un algoritmo de **Iteración de Políticas Modificada** (Puterman & Shin, 1978) es cualquier algoritmo que realice la evaluación de políticas un número finito de veces $k = 1, 2, \ldots, \infty$.

Profundizamos en ...

1. Introducción

2. Evaluación de Políticas (Predicción)

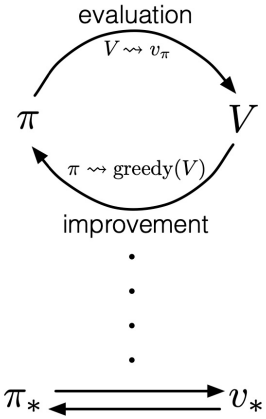
3. Mejorar la Política (Control)

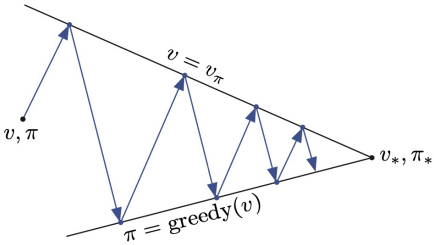
4. Iteración de Políticas
   - 4.1. Iteración de Políticas
   - 4.2. Iteración de Políticas Generalizada
   - 4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   - 5.1. Programación Síncrona
   - 5.2. Programación Asíncrona
   - 5.3. Problema de la Dimensionalidad

6. Referencias

Iteración de Políticas Generalizada





- Ambos procesos se estabilizan cuando se encuentra una política que es codiciosa con respecto a su propia función de evaluación

- **Evaluación de política** Estimar v_π
  - Cualquier algoritmo de evaluación de política
- **Mejora de política** Generar $\pi' \geq \pi$
  - Cualquier algoritmo de **mejora** de política

Profundizamos en ...

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Principio de optimalidad

- La optimalidad de un problema NO viene dado por la optimalidad de sus subproblemas.
  - $\max x + y$ s.a. $x + y \leq 1$ con $x, y \in \{0, 1\}$
    Tiene soluciones óptimas $(0, 1)$ y $(1, 0)$. Ambas suman 1.
  - Subproblema 1. Maximizar x con $x \in \{0, 1\}$. Solución óptima: $x = 1$
  - Subproblema 2. Maximizar y con $y \in \{0, 1\}$. Solución óptima: $y = 1$
  - La composición de soluciones parciales $x + y = 1 + 1 = 2$ ni siquiera cumple las restricciones.

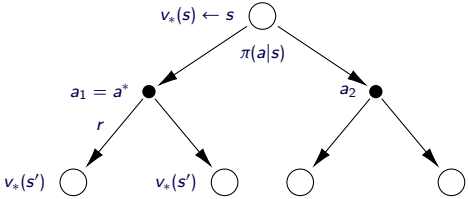
Principio de Optimalidad

- Una política óptima tiene la propiedad de que cualquiera que sea el estado inicial y la decisión inicial, las decisiones restantes deben constituir una política óptima en relación con el estado resultante de la primera decisión. (Bellman et al., 1957)
  - Si una solución es óptima, las subsoluciones son óptimas.

Teorema (Principio de Optimalidad)

- *Una política $\pi(a|s)$ alcanza el valor óptimo desde el estado s, $v_\pi(s) = v^*(s)$, si y solo si:*
  - *Cualquier estado s' es alcanzable desde s de forma óptima.*
  - *π alcanza el valor óptimo desde el estado s', $v_\pi(s') = v^*(s')$.*

Iteración de valores determinista



Si una trayectoria es óptima debe estar compuesta por subtrayectorias óptimas.

– *Principio de Optimalidad*

- Para **garantizar el principio de optimalidad**, cualquier política óptima debe poder subdividirse en dos componentes:
  - Una acción inicial óptima a^*.
  - Seguido de una política óptima desde sus estados sucesores S' (porque necesitamos un valor esperado).
- Si conocemos la solución a los subproblemas $v_*(s')$, entonces la solución $v_*(s)$ se puede encontrar mediante una **evaluación de un paso**:

$$v^*(s) \leftarrow \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v^*(s') \right)$$

Idea: aplicar estas actualizaciones de manera iterativa.

- Intuición: Comenzar con las recompensas finales y trabajar hacia atrás.
- Funciona incluso con MDPs estocásticos y con bucles.

Ejemplo: Camino más corto – I

$v_{k+1}(s) \leftarrow \text{máx}_{a \in A}\left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) v_k(s')\right] = \text{máx}_{a \in A}\left[R(s,a) + v_k(s')\right]$

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

- Problema: Encontrar el camino más corto hasta el objetivo **g**. $R(s,a) = -1$ y $\gamma = 1$
- v_1. Definimos $v_1(s) = 0 \ \forall s$
- v_2.
  - Estado terminal $(1,1)$: $v_1(1,1) = v_0(1,1) = 0$. No hay acciones
  - Estados no terminales $(i,j) \neq (1,1)$. $v_1(i,j) = \text{máx}_a[R_s^a + \sum_{s'} P(s'|s,a) v_0(s')] = -1 + 0 = -1$.
- v_2.
  - Estado terminal $(1,1)$: $v_2(1,1) = v_1(1,1) = 0$. No hay acciones
  - Adyacentes al terminal $v_2(i,j) = \text{máx}_a[R_s^a + \sum_{s'} P(s'|s,a) v_1(s')] = \text{máx}\{-1,-2\} = -1$.
  - Resto de estados: $v_2(s) = \text{máx}_{a \in A}[-1 + v_1(s')] = \text{máx}\, -2 = -2$

Ejemplo: Camino más corto – II

$v_{k+1}(s) \leftarrow \text{máx}_{a \in A}[-1 + v_k(s')]$

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

V_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

V_5

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

V_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

V_7

Iteración de valores

Algorithm 3 Iteración de valor para estimar $V \to v^*$

Entrada: Un pequeño umbral $\epsilon > 0$ que determina la precisión de la estimación.

1: Inicializar $V(s)$, para todo $s \in S^+$, arbitrariamente excepto que $V(terminal) = 0$.
2: **repeat**
3: $\quad \Delta \leftarrow 0$
4: $\quad$ **for** cada $s \in S$ **do**
5: $\quad\quad v \leftarrow V(s)$
6: $\quad\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$ # Alternativamente $\max_{a \in A}\left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s')\right)$
7: $\quad\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
8: $\quad$ **end for**
9: **until** $\Delta < \epsilon$

Salida: una política determinista, $\pi \approx \pi^*$, tal que $\pi(s) = \arg\max_a \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$

Iteración de valor: Lo mires por donde lo mires

La siguiente expresión se obtiene por distintos criterios:

$$v(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)\left[r + \gamma v(s')\right] \qquad \mathbf{v} \leftarrow \max_a \left[\mathbf{R}^a + \gamma \mathbf{P}^a \mathbf{v}\right]$$

- **El principo de optimalidad.**
  - Explicación anterior
- **Ecuación de optimalidad de Bellman.** $v_*(s) = \max_a \sum_{s',r} p(s',r|s,a)\left(r + \gamma v_*(s')\right]$
  - El algoritmo aplica una evaluación iterativa sobre la ecuación de optimalidad
- **Iteración de política modificada.**
  - Iteración de política modificada: **una** evaluación de la función valor y después cambia a política greedy.
    - $v_{\pi'}(s) = \max_{a\in\mathcal{A}} \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi(s')\right] = \begin{cases} 1. v_\pi(s) \leftarrow \sum_{a\in\mathcal{A}} \pi(a|s) \sum_{s',r} p(s',r|s,a)\left(r + \gamma v_\pi(s')\right) \\ 2. v_{\pi'}(s) = \max_{a\in\mathcal{A}} q_\pi(s,a) \end{cases}$

Desarrollo

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Profundizamos en ...

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Algoritmos Sincrónicos de Programación Dinámica:

- Son **Algoritmos Sincrónicos** los que usan dos tablas para resolver estos problemas:

Problema	Ecuación de Bellman	Algoritmo
Predicción	Ecuación de Expectativa de Bellman	Evaluación Iterativa de Políticas
Control	Ecuación de Expectativa de Bellman+ Mejora de Política Codiciosa	Iteración de Políticas
Control	Ecuación de Optimalidad de Bellman	Iteración de Valores

- Los algoritmos está basados en la **función valor de estado** $v_\pi(s)$ o $v_*(s)$
  - **Complejidad:** $O(mn^2)$ para m acciones y n estados.
    - *Evaluación Iterativa de Políticas:* $v^\pi(s) = \sum_{a\in A} \pi(a|s)\left[R_s^a + \gamma \sum_{s'\in S} P_{ss'}^a v^\pi(s')\right]$
      - Operaciones por estado s: $m \cdot \left[O(1) + \underbrace{n}_{\text{suma sobre } s'}\right] = O(mn)$.
      - Operaciones para todos los estados: $n \cdot O(mn) = O(mn^2)$.
    - *Iteración de Valores:* $v^*(s) = \max_{a\in A}\left[R_s^a + \gamma \sum_{s'\in S} P_{ss'}^a v^*(s')\right]$.
      - Operaciones por estado s: $\underbrace{m \cdot (O(1) + O(n))}_{\text{evaluar todas las acciones}} + \underbrace{O(m)}_{\text{calcular el máximo}} = O(mn)$.
      - Total para todos los estados: $n \cdot O(mn) = O(mn^2)$ por iteración.
- También se puede aplicar a la **función de valor de acción** $q_\pi(s,a)$ o $q^*(s,a)$.
  - **Complejidad:** $O(m^2n^2)$ por iteración.

Profundizamos en ...

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Programación Dinámica Asincrónica

Programación Dinámica Asincrónica

- Los métodos de programación dinámica descritos hasta ahora usan **respaldos sincrónicos**:
  - Es decir, todos los estados se respaldan en paralelo.
- La programación dinámica asincrónica **respalda estados individualmente**, en cualquier orden.
  - Para cada estado seleccionado, se aplica el respaldo apropiado.
- Puede reducir significativamente la computación.
- Garantiza la convergencia si todos los estados continúan siendo seleccionados.

Tres ideas simples para la programación dinámica asincrónica:

- Programación dinámica en el lugar. (*In-place*)
- Barrido priorizado. (*Prioritised sweeping*)
- Programación dinámica en tiempo real. (*Real-time*)

Programación Dinámica Asincrónica: *En el Lugar - In Place*

- La iteración de valores almacena **dos copias** de la función de valor:

$$\text{para todos } s \in S : v_{\text{nuevo}}(s) \leftarrow \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\text{viejo}}(s') \right)$$

$$v_{\text{viejo}} \leftarrow v_{\text{nuevo}}$$

- La **iteración de valores en el lugar** almacena **solo una copia** de la función de valor:

$$\text{para todos } s \in S : v(s) \leftarrow \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v(s') \right)$$

Programación Dinámica Asincrónica: *Barrido Priorizado*

Moore y Atkeson, 1993

- Utiliza la magnitud del **error de Bellman** para guiar la selección del estado:

$$\left| \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v(s') \right) - v(s) \right|$$

- **Respaldar** el estado con el mayor error de Bellman restante.
- **Actualizar** el error de Bellman de los estados afectados después de cada respaldo.
- **Requiere** conocimiento de las **dinámicas inversas** (estados predecesores).
- Puede implementarse eficientemente manteniendo una **cola de prioridades**.

Programación Dinámica Asincrónica: *Tiempo Real*

- Idea: seleccionar solo los estados que son relevantes para el agente.
- Usar la experiencia del agente para guiar la selección de estados
- Después de cada paso temporal (S_t, A_t, R_{t+1}) calculamos el backup del estado S_t:

$$v(S_t) \leftarrow \max_{a \in A} \left(R^a_{S_t} + \gamma \sum_{s' \in S} P^a_{S_t s'} v(s') \right)$$

Profundizamos en ...

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

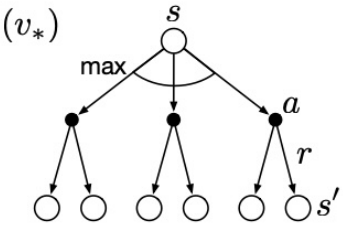
4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Backups Completos
Full-Width Backups

- La programación dinámica utiliza **backups completos**:
  - Para cada respaldo (sincrónico o asincrónico), se consideran todos los estados sucesores y acciones.
  - Utiliza el conocimiento de las transiciones del MDP y la función de recompensa.
- La PD es **efectiva** para problemas de tamaño medio (millones de estados).
- Para *problemas grandes*, sufre la **maldición de la dimensionalidad**:
  - El número de estados $n = |S|$ crece exponencialmente con el número de variables de estado.
  - Incluso un único backup puede ser demasiado costoso.
- Una solución es **mediante muestreo**: solo algunas trayectorias.



- Repite esta diagrama de respaldo desde los estados iniciales hasta los estados finales.

Desarrollo

1. Introducción

2. Evaluación de Políticas (Predicción)

3. Mejorar la Política (Control)

4. Iteración de Políticas
   4.1. Iteración de Políticas
   4.2. Iteración de Políticas Generalizada
   4.3. Iteración de Valor

5. Extensiones de la Programación Dinámica
   5.1. Programación Síncrona
   5.2. Programación Asíncrona
   5.3. Problema de la Dimensionalidad

6. Referencias

Referencias

- Bellman, R., Corporation, R., & Collection, K. M. R. (1957). *Dynamic Programming*. Princeton University Press. https://books.google.es/books?id=wdtoPwAACAAJ
- Moore, A. W., & Atkeson, C. G. (1993).Prioritized sweeping: Reinforcement learning with less data and less time. *Machine Learning*, *13*(1), 103–130.
- Puterman, M. L., & Shin, M. C. (1978).Modified policy iteration algorithms for discounted Markov decision problems. *Management Science*, *24*(11), 1127–1137.
- Silver, D. (2015). RL Course by David Silver - Lecture 2: Markov Decision Process. https://www.youtube.com/watch?v=lfHX2hHRMVQ
- Sutton, R., & Barto, A. (2018). *Reinforcement Learning, second edition: An Introduction*. MIT Press. https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf
- van Hasselt, H. (2018). Reinforcement Learning 3: Markov Decision Processes and Dynamic Programming. https://youtu.be/hMbxmRyDw5M?si=TIcGNDuuGetrxe-5